

QuantFundAgent

A quick catch-up on my thesis project

Luca Würker

June 1, 2026

The purpose of this document is to give a brief overview of the current state of my master thesis project as well as an outlook on future work and directions to take.

1 High level project description

My goal is to build a pipeline of LLM agents that mimick the workings within a quantitative hedge fund or proprietary trading firm as closely as possible. In the current version this is a pipeline of researching new features, forming a hypothesis about market behavior, selecting features and combining them into a strategy based on this hypothesis, running statistical tests and then allocating capital across the strategies.

Rough thesis goal. While much of the existing literature on agentic systems in finance focuses on using Large Language Models (LLMs) to analyze text sources—such as news or earnings call transcripts to mimic the discretionary behavior of fundamental investors [5], my goal is to design agents that function as a fully automated quantitative fund. While initial steps toward this direction have been explored [4, 3], these works primarily focus on discovering new alpha factors in a semi-automated manner rather than constructing and executing comprehensive, end-to-day trading strategies.

To bridge this gap, I intend to develop a complete agentic framework modeled after a quantitative fund’s operations. Using this framework, I will simulate a long-term backtest incorporating weekly agent meetings. During these sessions, the agents will dynamically update their strategies and rebalance the portfolio in order to replicate the decision-making processes of a quantitative fund.

2 The agent pipeline

At a high level, the system is organised as a sequence of specialised agents, each implemented as a LangGraph subgraph. An overview of the architecture can be seen in the diagrams in `quant_fund_agent/diagrams`.

Stage	Role
1	Factor Researcher — expand the alpha library from papers
2	Selector — form a hypothesis and pick factors
3	Architect — combine factors into a tradable strategy
4	Statistician — statistical accept/reject gate
5	Portfolio Manager — screen, allocate, monitor, rebalance

The data underpinning all of this are **10-second US large-cap equity bars** including all

market actions at the NASDAQ. At the current stage I use 50 stocks over a 3 month time period, but plan to extend the data to at least a year and potentially all the 500 S&P 500 stocks.

2.1 Factor Researcher

The preimplemented baseline consists of 86 alpha factors.¹ from the paper [2]. These are then appended through further new factors based on the agents research. Each factor is implemented as a class all inheriting from one base class `BaseFactor` in `quant_fund_agent/agents/factors/base.py`. The Factor Researcher mimics the research desk that reads academic and practitioner literature and tries to turn ideas into testable signals. In one *research session* (`quant_fund_agent/agents/factor_research/graph.py`) the agent:

1. selects papers from a predefined database of papers,
2. proposes several distinct factor ideas in a single brainstorming step,
3. generates full Python implementations of `BaseFactor` subclasses and
4. runs a single-factor IC backtest on every factor. This is a simple vectorised backtest and doesn't include transaction costs, slippage or any execution logic other than the plain signal.

Currently the paper database consists of only three papers in order to create a minimum working example, but will be grown extensively. Future extensions could also include free web access for the factor researcher agent.

The heavy steps (paper loading, code materialisation, panel-backed IC backtests, persistence) are exposed through an MCP server (`mcp/research_server.py`), so the research agent can call deterministic tooling without reloading the full intraday panel on every iteration.

2.2 Selector

Once the factor library is populated, the Selector agent (`quant_fund_agent/agents/selector/graph.py`) plays the role of a researcher proposing a tradeable idea. At the current stage every strategy has to be based on an idea about market behavior to avoid overfitting on noise as much as possible. A future test could also be to soften this restriction. The agent loads a compact summary of every backtested factor from `data/factors/factor_db.json` (via the `quant-catalog` MCP server), then uses an LLM to:

- formulate one clear trading hypothesis grounded in the available signals, and
- select the subset of factors that best support that hypothesis, with a short rationale. The IC of the single factors are not known to the selector to avoid a bias towards high IC factors, which could shrink the number of factors utilised and hinder the workflow from using factors like volatility that could have low stand-alone IC, but be important combined with other features.

The output—selected factor IDs plus the hypothesis text—is passed downstream to the Architect. At this stage the LLM is doing reasoning over structured metadata (IC, ICIR, category, description); no portfolio or execution decisions are made yet.

2.3 Architect

The Architect (`quant_fund_agent/agents/architect/graph.py`) is the main part of the strategy creation phase. He turns the Selector's hypothesis into a concrete strategy specification. It operates in a **refinement loop**:

¹I define an alpha factor as a formulaic calculation derived from available data that possesses predictive power (analogous to a “feature” in machine learning literature), rather than a risk factor in the traditional asset pricing sense.

1. **Design** — choose a model from a pre-implemented toolbox (linear models, tree ensembles, gradient boosting, or a static weighted baseline), set hyperparameters, position limits, and holding period.
2. **Fit and backtest** — fit on an in-sample window with an internal train/validation split; report in-sample metrics and diagnostics (e.g. train vs. validation R^2 for ML models). This is again a simplified vectorised backtest.
3. **Evaluate** — review metrics and trial history; either approve the specification or revise and try again. In case of a revision he has to keep track of the number of revisions that have been performed.

Model fitting and in-sample backtesting are delegated to the `quant-modeling` MCP server (`mcp/modeling_server.py`), which owns the heavy data panel server-side. Fitted ML models are persisted as joblib artifacts under `data/strategies/models/` and reloaded later for out-of-sample evaluation (`strategies/model_strategy.py`). Crucially, a held-out OOS slice of the data is *never* shown to the Architect; it is reserved exclusively for the Statistician.

Every trial is recorded in a trial history, which feeds into the deflated Sharpe ratio test—so a wider model search is correctly penalised for multiple testing.

In the current stage the architect agent can choose from previously mentioned machine learning models which are preimplemented. He is merely responsible for selecting the model and finding good (hyper-)parameters. A further extension could be to include give the agent more freedom to write completely new code in a sandbox environment and accepting it to the fund after passing sufficient tests.

2.4 Statistician

The Statistician (`quant_fund_agent/agents/statistician/graph.py`) acts as an independent risk and validation gate before any strategy enters the fund’s book. It runs a registry of statistical tests (`statistics/`); in the current MVP two tests are mandatory:

- **Deflated Sharpe ratio** [1] — adjusts for selection bias and non-normality using the Architect’s full trial history.
- **Out-of-sample backtest** — reloads the persisted model (without refitting) and evaluates performance on the held-out slice the Architect never saw. Again using a simplified vectorized backtest.

An LLM synthesises all test results into a final accept/reject decision, but a rule-based safety net forces rejection if any mandatory test fails outright. Optional tests can be added to the registry without changing the agent’s graph structure. A future agent that has been discussed but not implemented yet could be a further statistician researcher that reads papers and comes up with new statistical tests to validate strategies.

There is a covariance matrix consisting of the covariances between all the active strategies. For this (at the moment) no fixed lookback window but all available returns are used.

2.5 Portfolio Manager

Approved strategies are registered in the strategy database together with their per-bar portfolio return series (`persist_approved_strategy` in `pipeline.py`). Only then can the Portfolio Manager (`quant_fund_agent/agents/portfolio_manager/graph.py`) operate on a non-empty book.

The PM pipeline mirrors a real allocator’s workflow:

1. **Load universe** — read all non-retired strategies, correlation matrix, and covariance matrix from `StrategyDatabase`.
2. **Monitor** — for each live strategy, compare live performance to in-sample metrics; flag for

- review or retire if performance has broken (rule-based in *selector* mode, LLM-assisted in *active* mode).
3. **Screen** — apply personality-specific filters (minimum Sharpe, maximum drawdown, correlation caps) and pick a diversified roster.
 4. **Construct** — allocate capital using one of several methods implemented in `portfolio/construction.py` (equal weight, inverse volatility, minimum variance, mean–variance, maximum Sharpe, risk parity, hierarchical risk parity).
 5. **Finalise** — write a `PortfolioRecord` to the portfolio database and update each strategy’s deployment status (live, paused, flagged, retired).

Three pre-defined **personalities** (`portfolio/personalities.py`)—defensive, balanced, aggressive—encode different risk appetites. Multiple PMs can run in parallel, each producing its own portfolio record; alternatively, a **committee** module (`quant_fund_agent/agents/portfolio_manager/committee.py`) runs several PMs in “propose only” mode and aggregates their weight proposals into a single consensus allocation.

Screening and portfolio construction are exposed via `mcp/portfolio_server.py`; database mutations remain inside the agent graph because they rely on shared in-memory database handles (especially in the committee setting).

2.6 Tooling: MCP and reproducibility

Across all agents, I follow a consistent design pattern: **LLM steps reason in natural language**, while **deterministic quantitative work runs behind MCP servers** (Anthropic’s Model Context Protocol). Each domain—modeling, catalog, research, statistics, portfolio—has a thin stdio server, a shared `*_service.py` implementation, and a synchronous client facade. The test suite (`tests/test_mcp*.py`) verifies that the MCP path and an in-process fallback return identical results, which keeps the system debuggable without sacrificing the separation of concerns.

This keeps memory-heavy objects (the full intraday panel, cached factor signals) on the server side rather than in every agent process.

3 Current state

The project is at a working **MVP** stage: the full vertical slice runs end-to-end—from factor research through strategy approval to portfolio allocation—via `run_fund.py`. Individual agents are first implementations; prompts and decision logic are intentionally simple and will be deepened in future work. Most tests on the current setup have shown that most of the created strategies do not pass the current statistician agents. So I will need to work on prompt and agent quality.

What is *not* yet the main empirical contribution is the **long-horizon simulation** described above: daily mark-to-market, weekly research sessions, repeated strategy pipelines, and periodic PM rebalances over roughly two months of 10-second data. The pipeline functions and incremental database APIs (e.g. `StrategyDatabase.append_returns`) are already written with that loop in mind; running and analysing it is the immediate next milestone.

4 Outlook and future directions

- **Benchmarks:** Currently no benchmarks have been implemented. In order to judge the quality of the agents output, I plan to implement the following benchmarks:
 - factor mining vs. factor research: Generate a number of alpha factors by simply combining known arithmetic and time series analysis operations on the available data. Compare

performance of subsequent steps with this brute force factors to those using the ones generated by the researchers

- strategy architect vs. brute force Machine Learning: Use a random forest (or other standard ML methods) on all available factors to create the best strategy(s), compare performance to the strategies created by the architect
- turn on/off different stages in the pipeline (e.g. always selecting n random factors instead of using the selector agent) to verify the usefulness of using agents in every single step
- Implement and report the scheduled multi-month backtest using the existing `pipeline.py` stage functions.
- Evaluate portfolio-level outcomes (Sharpe, turnover, drawdown) and process metrics (factor acceptance rate, strategy survival rate, PM turnover).
- **Deepen the single agents**
 - Substantially extend the database of available papers for the factor researcher agent. Potentially grant access to online search to come up with even more unconventional factors
 - Give architect freedom to write completely new code (in sandbox environment first) by himself
 - add more statistical tests to the statistician, give him freedom to choose which ones to apply, or even come up with new ones
- Extend dataset to several months and more underlyings
- Currently only the PM agent allows several instances that discuss, add different instances of agents to all levels and test the value of having them debate
- Long term: Turn this into a nice open-source project allow for different data vendors to create reusability for a broad audience (make it publishable?)

References

- [1] David H. Bailey and Marcos López de Prado. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014.
- [2] Zura Kakushadze. 101 formulaic alphas. *Wilmott*, 2016(84):72–80, 2016.
- [3] Saizheng Li, Shicheng Sun, Jialu Xue, Boyu Yang, Zidong Zhou, and John Zhang. Alpha-gpt: Human-ai interactive alpha mining for quantitative investment. *arXiv preprint arXiv:2308.00016*, 2023.
- [4] Boyi Wang, Hongyang Wang, Wei Chen, Yitong Chang, Shuo Sheng, Qi Liu, and Lingling Xu. Quantagent: Seeking holy grail in trading by self-improving large language model. *arXiv preprint arXiv:2402.11137*, 2024.
- [5] Xieyu Wang, Sifan Yang, Dawei Zhang, Incheon Chon, Edward Zhao, Susan Zhou, Derek Yang, Yusuf Ahmed, and David Liu. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2312.04902*, 2023.